5 Roteiro 05 – CRUD Completo com Flask

Objetivos

- Implementar operações CRUD completas;
- Manipular dados em memória;
- Trabalhar múltiplas rotas Flask;
- Entender fluxo de atualização e remoção de dados;
- Evoluir aplicação existente sem alterar estrutura base.

Resultado Final Esperado

Ao final deste roteiro, você será capaz de:

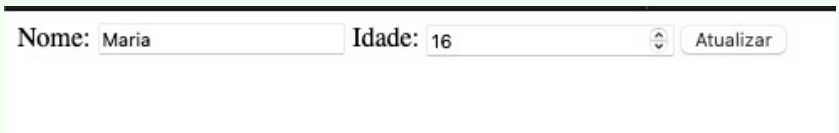
- Cadastrar alunos (Create);
- Listar alunos (Read);
- Editar alunos (Update);
- Remover alunos (Delete);
- Manipular dados dinamicamente em uma aplicação web.

A aplicação continuará utilizando Flask + HTML, porém agora com operações completas sobre os dados.

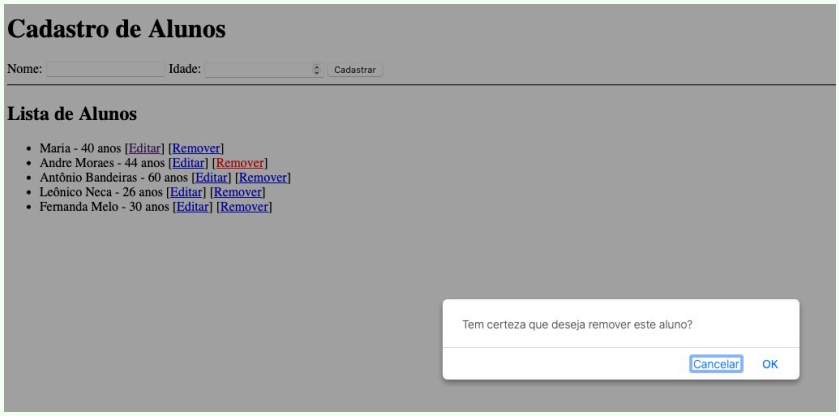
Visualização das funcionalidades implementadas:

- **Listagem de alunos:**

- **Edição de aluno:**



- **Remoção de aluno:**



O que observar nas interfaces:

- A listagem apresenta todos os alunos cadastrados;
- Cada item possui opções de edição e remoção;
- A edição permite atualizar os dados existentes;
- A remoção exclui o registro da aplicação;
- Todas as ações refletem imediatamente na interface.

Introdução

No roteiro anterior, implementamos o cadastro e listagem de alunos.

Agora iremos evoluir a aplicação para permitir:

- Editar dados existentes;
- Remover registros;
- Atualizar informações dinamicamente.

Com isso, completamos o ciclo CRUD (Create, Read, Update, Delete).

Parte 1 – Adicionando Identificador

Atualize a classe `Aluno` para incluir um identificador único:

```
class Aluno:
    contador = 1

    def __init__(self, nome, idade):
        self.id = Aluno.contador
        self.nome = nome
        self.idade = idade
        Aluno.contador += 1
```

Explicação:
Agora cada aluno possui um identificador único (id).
Isso é necessário para:

- Editar um aluno específico;
- Remover um aluno específico;

Sem esse identificador, não conseguimos diferenciar os registros.

Parte 2 – Implementando Remoção

No `app.py`, adicione:

```
@app.route("/remover/<int:id>")
def remover_aluno(id):
    global alunos
    alunos = [aluno for aluno in alunos if aluno.id != id]
    return redirect("/")
```

Explicação:

1. Recebemos o id pela URL;
2. Filtramos a lista removendo o aluno correspondente;
3. Atualizamos a lista em memória;
4. Redirecionamos para a página principal.

Parte 3 – Implementando Edição

Adicione duas rotas:

```
# Rota para carregar os dados de um aluno
@app.route("/editar/<int:id>")
def editar_aluno(id):
    aluno = next((aluno for aluno in alunos if aluno.id == id), None)
    if aluno:
        return render_template("editar.html", aluno=aluno)
    return redirect("/")


# Rota para atualizar um aluno
@app.route("/atualizar/<int:id>", methods=["POST"])
def atualizar_aluno(id):
    aluno = next((aluno for aluno in alunos if aluno.id == id), None)
    if aluno:
        aluno.nome = request.form["nome"]
        aluno.idade = int(request.form["idade"])
    return redirect("/")
```

Explicação:

- A rota `/editar` carrega os dados do aluno;
- A rota `/atualizar` recebe os dados atualizados;
- O objeto é modificado diretamente na memória;
- A página é recarregada com os dados atualizados.

Parte 4 – Atualizando o HTML

No `index.html`, adicione:

```
<h2>Lista de Alunos</h2>

<ul>
    {% for aluno in alunos %}
    <li>
        {{ aluno.nome }} - {{ aluno.idade }} anos
        [<a href="/editar/{{ aluno.id }}">Editar</a>]
        [<a href="/remover/{{ aluno.id }}"
            onclick="return confirm('Tem certeza que
            deseja remover este aluno?')">Remover</a>]
    </li>
    {% endfor %}
</ul>
```

Explicação:
Agora cada item possui ações:

- Editar → abre formulário de edição;
- Remover → exclui o aluno.

Parte 5 – Criando editar.html

```
<form method="POST" action="/atualizar/{{ aluno.id }}">
    Nome:
    <input type="text" name="nome" value="{{ aluno.nome }}">

    Idade:
    <input type="number" name="idade" value="{{ aluno.idade }}">

    <button type="submit">Atualizar</button>
</form>
```

Explicação:

- O formulário já vem preenchido;
- Os dados são enviados novamente via POST;
- O backend atualiza o registro existente.

Parte 6 – Testando CRUD

Execute:

```
python app.py
```

Teste:

- Criar alunos;
- Editar alunos;
- Remover alunos;

Observe:
Os dados ainda estão em memória — ao reiniciar o sistema, tudo será perdido.

Resumo do CRUD

- Create → adicionar aluno;
- Read → listar alunos;
- Update → editar aluno;
- Delete → remover aluno;

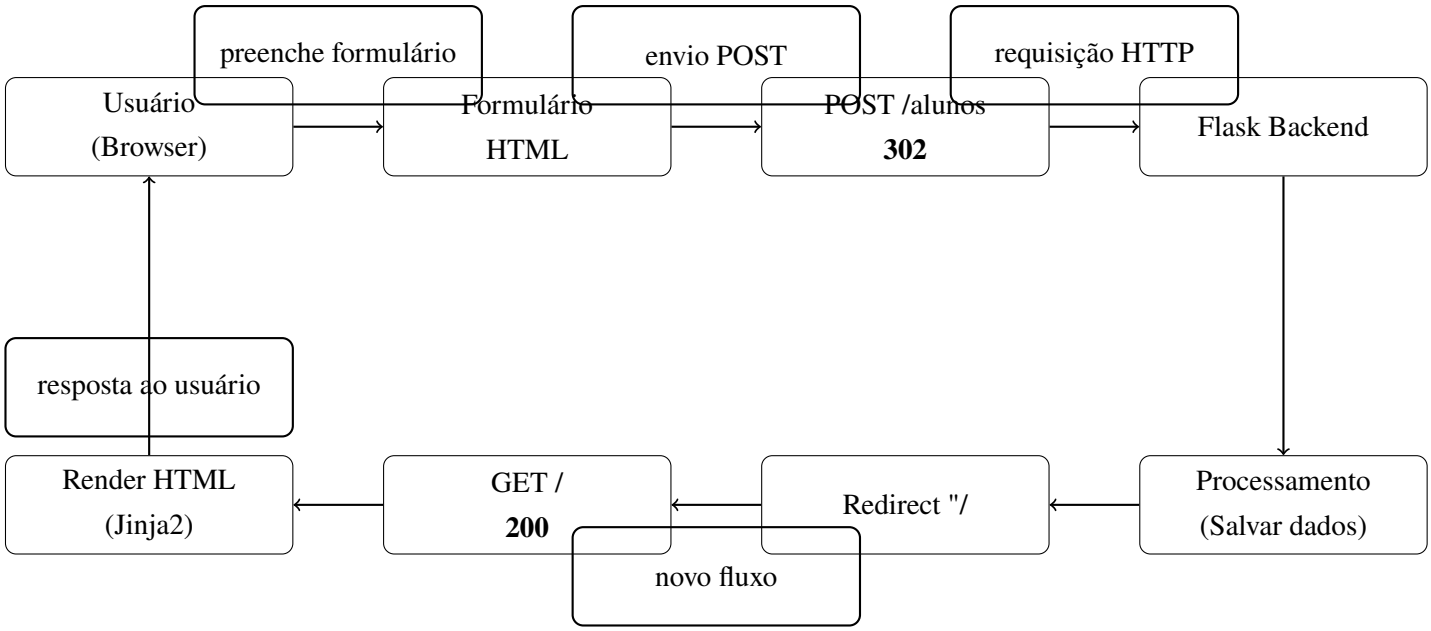
Interpretação dos Códigos de Requisição

Compreendendo os Códigos HTTP no Sistema

Código / Método	O que é	O que acontece no sistema
GET 200 (OK)	Requisição bem-sucedida para obter dados ou páginas.	A página é carregada normalmente no navegador, com os dados atualizados enviados pelo servidor.
POST 302 (Redirect)	Envio de dados seguido de redirecionamento.	O servidor processa os dados (ex: cadastro ou atualização) e redireciona o usuário para outra rota, evitando reenvio do formulário.
GET após 302	Nova requisição automática após redirecionamento.	A página é recarregada já com os dados atualizados, refletindo a operação realizada anteriormente.
GET 404 (Not Found)	Requisição para uma rota inexistente.	O servidor não encontra a rota solicitada (ex: erro de digitação na URL ou rota não implementada).
GET (Recursos automáticos)	Requisições feitas automaticamente pelo navegador.	O navegador tenta carregar arquivos como ícones (favicon). Mesmo com erro 404, isso não afeta o funcionamento da aplicação.
Fluxo POST → 302 → GET	Padrão de funcionamento de formulários web.	O sistema evita duplicação de envio e garante atualização correta da interface após cada operação.

Dica: Sempre que você interage com a aplicação, observe o console. Cada ação gera uma sequência de requisições HTTP que revelam o funcionamento interno do sistema.

Fluxo das Requisições na Aplicação



Entrega

- CRUD funcionando completamente;
- Interface atualizada;
- Código organizado;
- Versionamento com Git.